

Linux & Networking SRE Mastery Handbook

Audience: SREs targeting high-scale product companies (PhonePe, Razorpay, CRED, Netflix).
Scope: Kernel fundamentals → live packet-level incident triage.
Convention: Every operational command uses the five-part production format below.

Table of contents

- 1. Module 1 — Linux OS & core architecture
- 2. Module 2 — Essential commands & triage tooling
- 3. Module 3 — Networking fundamentals (OSI → sockets)
- 4. Module 4 — SRE network diagnostic toolkit
- 5. Module 5 — Top 30 interview questions
- 6. Module 6 — 14-day hands-on study plan

Module 1: The Linux Operating System & Core Systems Architecture

Kernel space vs user space

Layer	Ring	What runs here	SRE relevance
Kernel space	Ring 0	Scheduler, VM, VFS, TCP/IP stack, drivers	Latency spikes, soft lockups, D state, conntrack, drops
User space	Ring 3	Applications, language runtimes, agents	Thread pools, FD leaks, misconfigured ulimits

System calls are the controlled gateway from Ring 3 → Ring 0. Tracing tools (eBPF, `strace`, `audit`) hook `sys_enter` / `sys_exit` tracepoints to observe syscall rate, errors, and latency without recompiling the kernel.

Context switches occur when the CPU moves from one runnable entity to another (process/thread). High **cswch/s** with flat CPU can mean lock contention, I/O wait, or oversubscribed workers—not necessarily “high CPU.”

Process & thread management¶

State	Symbol	Meaning	Outage signal
Running	R	On CPU or runnable	CPU-bound hot loops
Interruptible sleep	S	Waiting (I/O, lock, poll)	Normal idle; huge counts → backlog
Uninterruptible sleep	D	Blocked in kernel (often disk/ NFS)	Load avg ↑ while CPU low
Zombie	Z	Exited but not reaped	Parent bug; PID exhaustion risk
Stopped	T	Job control / debugger	Deployments stuck

PID 1 (systemd) reaps orphaned children, mounts namespaces, and owns default cgroup hierarchies. If PID 1 is unhealthy, everything downstream looks “mysteriously broken.”

`fork()` duplicates the calling process (copy-on-write). `execve()` replaces the address space with a new program. Shell pipelines and container entrypoints are literally `fork` + `execve` choreography.

The `/proc` virtual filesystem¶

`/proc` is not on disk—it is live kernel state exported as files.

Path	What it tells you
<code>/proc/cpuinfo</code>	Model, cores, flags (AES-NI, etc.)
<code>/proc/meminfo</code>	Detailed RAM breakdown (see Module 2 <code>free</code>)
<code>/proc/sys/net/</code>	Tunables: <code>ipv4/tcp_*</code> , <code>core/somaxconn</code> , <code>conntrack</code>
<code>/proc/<pid>/fd/</code>	Open FDs (sockets, files, pipes)
<code>/proc/<pid>/status</code>	<code>VmRSS</code> , <code>Threads</code> , <code>FDSize</code> , <code>State</code>
<code>/proc/<pid>/net/tcp</code>	Socket table in hex (local/remote IP:port, state)

Reading `/proc/<pid>/net/tcp` : Columns include local address, remote address, state. State `01` = ESTABLISHED, `0A` = LISTEN (see kernel `tcp_states`).

```
# Decode listening sockets for PID 1234
sudo ls -l /proc/1234/fd | grep socket
sudo cat /proc/1234/net/tcp
grep -i somaxconn /proc/sys/net/core/somaxconn
```

File descriptors & inodes¶

Linux “**everything is a file**”: regular files, directories, sockets, pipes, epoll fds, timerfds.

Limit	Scope	Config
Soft ulimit	Per-process, enforceable at runtime	<code>ulimit -n , /etc/security/limits.conf</code>
Hard ulimit	Ceiling for soft limit	root can raise soft up to hard

Too many open files during peak traffic is almost always: connection leak, missing `close()` , or default `1024` `ulimit` on a payment API.

```
# Per-service limits (systemd)
systemctl show payment-api -p LimitNOFILE
cat /proc/$(pgrep -f payment-api | head -1)/limits | grep "open files"
```

Inodes index metadata. You can have free **blocks** but zero free **inodes** → creates fail with “No space left on device.”

Memory mechanics¶

Term	Meaning
Virtual memory (VSZ)	Address space reserved
RSS	Pages actually in RAM
Page cache	File-backed cache (reclaimable)
Dirty pages	Modified cache not yet flushed to disk
Swap	Paged-out anonymous memory—latency cliff on burst

OOM Killer triggers when reclaim fails. It scores processes via `oom_score` ; SREs protect critical daemons with `oom_score_adj` (-1000 to +1000, lower = safer).

```
# Protect kubelet (example)
echo -500 | sudo tee /proc/$(pgrep kubelet)/oom_score_adj
cat /proc/$(pgrep kubelet)/oom_score
```

Callout — Bar Raiser expectation: Strong candidates explain why `available` in `free` matters more than `free` column, and why `D` state inflates load average without consuming CPU.

Module 2: Essential Linux Commands & Triage Tooling¶

Command reference: `uptime` ¶

1. Command name & plain English meaning

Shows how long the system has been running and the **load averages** (1/5/15 minutes)—a smoothed count of threads wanting CPU or stuck in uninterruptible I/O.

2. Most frequent production flags

- (no flags) — default load averages - `-p` — pretty-print uptime (some distros)

3. Concrete command examples

```
uptime
uptime -p
```

4. Interpreting the output (outage triage)

Compare **load average** to **CPU count** (`nproc`). If 1-min load is 32 on 16 cores, you are saturated or I/O blocked. Pair with `mpstat` and `ps` for **D** state.

5. Real-world production war story

Payment reconciliation batch started at 02:00; load hit 40 on 16-core DB nodes while CPU was ~15%. Root cause: NFS-backed storage put dozens of threads in **D state**—not CPU. Moved scratch to local SSD; load normalized in minutes.

Command reference: `top` ¶

1. Command name & plain English meaning

Interactive **process table** sorted by CPU by default—first screen during “something is slow.”

2. Most frequent production flags

- `-b` — batch mode (for logs) - `-H` — show threads - `-p PID` — monitor specific PIDs - `-o %MEM` — sort by memory

3. Concrete command examples

```
top -H -p $(pgrep -f payment-api)
top -b -n 1 | head -30
```

4. Interpreting the output (outage triage)

Watch **%CPU**, **RES**, **S column** (state), **TIME+**. One thread at 100% → hot loop; many threads in **S** → wait on lock/I/O.

5. Real-world production war story

API p99 doubled; `top -H` showed 200 threads blocked in **S** on same Java process—thread pool queue full. Increased pool after fixing downstream Redis timeouts (real bottleneck).

Command reference: `htop` ¶

1. Command name & plain English meaning

User-friendly `top` with per-CPU bars, tree view, and mouse support—faster situational awareness in war rooms.

2. Most frequent production flags

- `-p PID` — highlight PIDs - `-t` — tree view (install variant dependent) - `-d SEC` — delay between updates

3. Concrete command examples

```
htop -p $(pgrep -f nginx | tr '\n' ',')
htop -d 2
```

4. Interpreting the output (outage triage)

Per-CPU saturation vs single-core hot spot. Tree view exposes **parent/child** (worker master → workers).

5. Real-world production war story

During Diwali peak, one nginx worker at 100% while others idle— `SO_REUSEPORT` misconfigured. `htop` tree made skew obvious; fixed upstream hash policy.

Command reference: `mpstat -P ALL 1 1`

1. Command name & plain English meaning

Per-CPU utilization from `sysstat`—separates user/system/iowait/steal—essential on NUMA and noisy-neighbor VMs.

2. Most frequent production flags

- `-P ALL` — every CPU - `1` — 1-second interval - `-u` — CPU report (default) - `-I SUM` — interrupt stats

3. Concrete command examples

```
mpstat -P ALL 1 5
mpstat -I SUM 1 3
```

4. Interpreting the output (outage triage)

`%iowait` high → disk/network storage. `%steal` high on cloud → hypervisor contention. One CPU at 100% others idle → pinning/lock issue.

5. Real-world production war story

Kafka broker “slow” but CPU looked fine on average— `mpstat` showed **single core pegged** on IRQ-heavy NIC queue. RSS/RPS tuning spread load.

Command reference: `pidstat -u 1 1`

1. Command name & plain English meaning

Per-process CPU over time—better than one-shot `ps` for proving which PID burned CPU during an incident window.

2. Most frequent production flags

- `-u` — CPU - `-r` — memory - `-w` — context switches - `-p PID` — filter - `1` — interval seconds

3. Concrete command examples

```
pidstat -u 1 10
pidstat -w -p $(pgrep java) 1 5
```

4. Interpreting the output (outage triage)

Watch `%CPU`, `cswch/s` (voluntary), `nvcschw/s` (involuntary). High involuntary → preemption/CPU starvation.

5. Real-world production war story

Microservice regression after deploy: `pidstat` tied 80% CPU to one PID; roll back while strace confirmed tight retry loop on 404 from config service.

Command reference: `free -m` ¶

1. Command name & plain English meaning

Reports **memory and swap** in megabytes—interpret `available`, not just `free`.

2. Most frequent production flags

- `-m` — megabytes - `-h` — human readable - `-w` — wide view (separate cache columns on newer procs)

3. Concrete command examples

```
free -m
free -wh
```

4. Interpreting the output (outage triage)

`available` ≈ memory for new workloads without swapping. Low available + rising `si/so` in `vmstat` → swap storm incoming.

5. Real-world production war story

Node flapped during batch job; `free` showed 200MB ‘free’ but **only 150MB available**—page cache was huge but not reclaimable fast enough; OOM killed sidecar first.

Command reference: `vmstat 1` ¶

1. Command name & plain English meaning

System-wide counters each second: processes, memory, swap, I/O, CPU—classic “is it CPU, memory, or disk?” triage.

2. Most frequent production flags

- `1` — interval - `-S M` — megabyte units - `-w` — wide

3. Concrete command examples

```
vmstat 1 20
vmstat -w 1 10
```

4. Interpreting the output (outage triage)

`r` runnable queue; `b` uninterruptible; `si/so` swap in/out; `wa` I/O wait CPU%; `id` idle.

5. Real-world production war story

Checkout latency: `vmstat` showed `b=12` constantly—disk/NFS **D state**. Storage team found saturated array; not application GC.

Command reference: `df -h` ¶

1. Command name & plain English meaning

Shows **filesystem space usage** per mount—first check when writes fail.

2. Most frequent production flags

- `-h` — human sizes - `-T` — fstype - `-i` — **inode** usage (critical) - `--`

output=source,fstype,size,used,avail,pcent,target

3. Concrete command examples

```
df -hT
df -i /var
```

4. Interpreting the output (outage triage)

100% on any mount stops writes. `df -h` **OK but full errors** → check `df -i` and `lsdf +L1`.

5. Real-world production war story

Log shipper stopped; `/var` at 45% blocks but **inodes 100%**—millions of tiny temp files. Cleanup + log rotation fix.

Command reference: `iostat -xz 1` ¶

1. Command name & plain English meaning

Disk I/O statistics—find saturated devices (`%util` , `await`).

2. Most frequent production flags

- `-x` — extended stats - `-z` — omit idle devices - `-d` — device only - `1` — interval

3. Concrete command examples

```
iostat -xz 1 10
iostat -xmd 1 5
```

4. Interpreting the output (outage triage)

`%util` ~**100** → device saturated. `await` ms high → latency. `r/s / w/s` with high `await` → backlog.

5. Real-world production war story

DB checkpoint storm: `nvme0n1` **await 80ms**, util 99%. Throttled checkpoint + moved WAL to faster volume; p99 query latency recovered.

Command reference: `iostat -oP` ¶

1. Command name & plain English meaning

Shows **which processes** drive disk I/O—`iostat` needs root or capabilities.

2. Most frequent production flags

- `-o` — only active I/O - `-P` — processes (not threads) - `-a` — accumulated I/O - `-d SEC` — delay

3. Concrete command examples

```
sudo iotop -oP -d 2
sudo iotop -oPa
```

4. Interpreting the output (outage triage)

Identify unexpected `java` or `rsync` during peak. `DISK WRITE` spikes during deploy often = heap dumps or bad logging.

5. Real-world production war story

Mystery SSD wear: `iotop` showed backup agent scanning entire disk hourly—disabled on prod tier.

Command reference: `lsuf +L1` ¶

1. Command name & plain English meaning

Lists **open files**; `+L1` finds **deleted files still held open** (space not reclaimed).

2. Most frequent production flags

- `-p PID` — process - `-i :443` — network - `+L1` — link count <1 (deleted but open) - `-nP` — no DNS/port names

3. Concrete command examples

```
sudo lsuf +L1 | head
sudo lsuf -p $(pgrep java) | wc -l
```

4. Interpreting the output (outage triage)

Deleted file with large size and `PID` still writing → `df` **shows used space** until process closes FD or restarts.

5. Real-world production war story

“Disk full” alert but `df` 50%: `/var/log/app.log (deleted)` held by app at 200GB. Rolling restart freed space instantly.

Command reference: `strace` ¶

1. Command name & plain English meaning

Traces **system calls**— proves what a process actually asks the kernel (connect, read, futex, etc.).

2. Most frequent production flags

- `-p PID` — attach - `-f` — follow threads - `-e trace=network,file` — filter - `-T` — syscall time - `-tt` — timestamps - `-y` — print paths

3. Concrete command examples

```
sudo strace -fp $(pgrep -f payment-api) -e trace=network -T -tt
sudo strace -c -p PID # summary
```

4. Interpreting the output (outage triage)

Stuck on `connect()` → network/DNS. **EAGAIN on read** → non-blocking wait.

`futex(... FUTEX_WAIT` → lock/thread pool.

5. Real-world production war story

Intermittent 5s timeouts: strace showed `connect` taking **5003ms** to internal LB—SYN not answered. Network team found security group drift, not app thread pool.

Command reference: `ps aux --sort=-%mem` ¶

1. Command name & plain English meaning

Snapshot of **all processes**—sort by memory to find OOM candidates.

2. Most frequent production flags

- `aux` — all users, BSD format - `--sort=-%mem` — top memory - `-L` — threads - `-o pid,ppid,cmd,%mem,rss` — custom columns

3. Concrete command examples

```
ps aux --sort=-%mem | head -20
ps -eo pid,stat,cmd | awk '$2 ~ /D/ {print}'
```

4. Interpreting the output (outage triage)

`STAT` contains `D` → I/O blocked. `RSS` large + `Ssl` java → heap/off-heap pressure.

5. Real-world production war story

Pre-OOM: top 3 Java pods at 92% memory limit—HPA scaled on CPU only; added memory-based scaling and fixed leak in cache map.

Command reference: `systemctl` ¶

1. Command name & plain English meaning

Controls **systemd units**—how services start, restart, and log on modern Linux.

2. Most frequent production flags

`status UNIT` — health + recent logs `restart UNIT` — bounce service `mask UNIT` — prevent start (break-glass) `show UNIT -p ActiveState,SubState`

3. Concrete command examples

```
systemctl status nginx
systemctl restart payment-api
systemctl show kubelet -p LimitNOFILE,MemoryMax
```

4. Interpreting the output (outage triage)

`failed (Result: exit-code)` → check `journalctl -u`. `activating (auto-restart)` → crash loop. `masked` → intentional disable.

5. Real-world production war story

Deploy “succeeded” but traffic 503: unit `ActiveState=activating`—bad ExecStart path. `systemctl status` showed exit 203/EXEC in 2 seconds.

Command reference: `journalctl` ¶

1. Command name & plain English meaning

Reads **systemd journal**—kernel, service, and structured logs with powerful filters.

2. Most frequent production flags

- `-u UNIT` — unit logs `-k` — kernel `-p err` — priority `--since "1 hour ago"` `-f` — follow `-o json-pretty`

3. Concrete command examples

```
journalctl -u payment-api --since "2026-10-01 18:00" -p warning
journalctl -k -b -1 # previous boot kernel
```

4. Interpreting the output (outage triage)

Correlate **timestamps** with metrics. `--boot -1` for post-mortems after reboot. Kernel `nf_conntrack: table full` appears here with `-k`.

5. Real-world production war story

Midnight incident: `journalctl -k` showed **NIC link flaps** aligned with TCP retrans spike—datacenter switch port error counters confirmed bad cable.

Module 3: Linux Networking Fundamentals (OSI to Socket Layer) ¶

Journey of a packet (curl → wire) ¶

1. **User space:** Application calls `write()` on a connected TCP socket; libc wraps `sendmsg()` syscall.
2. **Kernel TCP:** Segments data, applies congestion window, enqueues `sk_buff` chains in socket send buffer.
3. **IP layer:** Routing lookup (`FIB`), TTL, DF bit for PMTU, optional XFRM if IPsec.
4. **NIC driver:** DMA from ring buffer; `ethtool -S` exposes drops if ring overrun.
5. **Reverse path:** NIC → NAPI polling → netfilter/conntrack → socket receive queue → `read()` in app.

TCP internals for SRE ¶

3-way handshake: `SYN` → `SYN-ACK` → `ACK`. **Teardown:** `FIN` / `ACK` exchange (often four segments with simultaneous close variants).

State	Meaning	Action
LISTEN	Accept queue active	Check <code>ss -lnt</code> Recv-Q vs <code>somaxconn</code>
ESTABLISHED	Data flow	If slow, check RTT/retrans
TIME_WAIT	Closed side waits 2×MSL	Normal on high churn clients; tune only with measurement
CLOSE_WAIT	Peer sent FIN; app never closed	Application FD leak

Buffers & congestion: Send/receive windows, window scaling (`TCP window scaling` option), Reno/Cubic/BBR behavior. **Zero-window** probes in `tcpdump` mean receiver starved.

DNS architecture¶

Resolution path (glibc): `/etc/nsswitch.conf` (`hosts: files dns`) → `/etc/hosts` → `/etc/resolv.conf` (or `systemd-resolved` via stub `127.0.0.53`).

Record	Purpose
A / AAAA	IPv4 / IPv6 address
CNAME	Alias (cannot coexist with other data at same name)
PTR	Reverse lookup
SRV	Service discovery
TXT	SPF, verification, feature flags

TTL caching: Stale records after cutover cause “split brain” clients. **EDNS0** enables larger responses; truncation → fallback to TCP.

Production tip: Always compare `dig @resolver` vs `dig @authoritative` during DNS incidents.

Module 4: The SRE Network Diagnostic Toolkit¶

Command reference: `ss` ¶

1. Command name & plain English meaning

Modern **socket statistics**—replaces `netstat` , faster on machines with tens of thousands of connections.

2. Most frequent production flags

- `-t` TCP - `-u` UDP - `-l` listening - `-n` numeric - `-p` process - `-s` summary - `-i` internal TCP info

3. Concrete command examples

```
ss -tulnp
ss -s
ss -ltn '( sport = :8080 )'
```

4. Interpreting the output (outage triage)

Recv-Q on LISTEN = completed handshake queue backlog. **Send-Q** high on ESTAB → app not reading or network stuck.

5. Real-world production war story

SYN flood symptoms: `ss -s` shows `synrecv` high; **Recv-Q** on listener at `somaxconn`. Raised backlog + fixed app accept loop.

Command reference: `netstat` ¶

1. Command name & plain English meaning

Legacy **socket listing**—still seen on older runbooks; prefer `ss` on RHEL8+/Ubuntu 22+.

2. Most frequent production flags

- `-tulnp` — TCP/UDP listen numeric process - `-s` — protocol summaries - `-c` — continuous

3. Concrete command examples

```
netstat -tulnp
netstat -s | grep -i listen
```

4. Interpreting the output (outage triage)

Same semantics as `ss`; if numbers disagree, check namespace (`ip netns exec`).

5. Real-world production war story

Vendor script required `netstat`; used it in init container, migrated to `ss` on host for speed at 80k connections.

Command reference: `dig +trace` ¶

1. Command name & plain English meaning

DNS lookup with delegation tracing—shows which nameserver answers at each level.

2. Most frequent production flags

`+trace` — iterative path `+short` — answer only `@server` — force resolver `+ttlunits` — human TTL

3. Concrete command examples

```
dig +trace api.bank.com
dig @8.8.8.8 +short A api.bank.com
dig +dnssec +multi api.bank.com
```

4. Interpreting the output (outage triage)

Compare **ANSWER vs AUTHORITY** sections. **Unexpected CNAME chains** add latency. **SERVFAIL** → upstream or lame delegation.

5. Real-world production war story

Post-migration, 1% clients hit old IP: `dig +trace` showed stale NS at registrar—not app bug.

Command reference: `nslookup` ¶

1. Command name & plain English meaning

Interactive **DNS query** tool—quick checks; less script-friendly than `dig`.

2. Most frequent production flags

- `type=MX|TXT|SRV` - `server IP` — set resolver

3. Concrete command examples

```
nslookup api.bank.com
nslookup -type=SRV _https._tcp.api.bank.com
```

4. Interpreting the output (outage triage)

Non-authoritative vs authoritative answers in header. Timeout → firewall blocking UDP/53.

5. Real-world production war story

Corporate VPN split-DNS: `nslookup` returned internal IP while public `dig @1.1.1.1` showed public—explained “works in office, fails from prod.”

Command reference: `resolvectl status` ¶

1. Command name & plain English meaning

systemd-resolved view—stub resolver, DNS servers per interface, routing domains.

2. Most frequent production flags

`status` — full picture `query HOST` — test resolution path `flush-caches` — after DNS cutover (careful)

3. Concrete command examples

```
resolvectl status
resolvectl query api.bank.com
```

4. Interpreting the output (outage triage)

Look for **Link-specific DNS** vs global. **DNSSEC setting**. Misconfigured `~.` **domain** routes all queries wrong.

5. Real-world production war story

EKS node DNS flakes: `127.0.0.53` stub mis-forwarded to VPC resolver—`resolvectl status` showed stale upstream after DHCP renew.

Command reference: `ping` ¶

1. Command name & plain English meaning

ICMP echo reachability and RTT—not a substitute for TCP/443 health.

2. Most frequent production flags

- `-c N` — count - `-i SEC` — interval - `-s SIZE` — payload (MTU tests) - `-M do` — don't fragment - `-I iface` — source interface

3. Concrete command examples

```
ping -c 5 10.0.1.1
ping -s 1472 -M do -c 3 gateway.bank.com
```

4. Interpreting the output (outage triage)

100% loss → routing/firewall/ICMP blocked. **Frag needed** at size 1472 → **PMTU < 1500**. **Partial loss** → congestion or policer.

5. Real-world production war story

“Host up but app down”: ping OK (ICMP allowed) but **:443 filtered**—explained to leadership why ping green ≠ service healthy.

Command reference: `tracert` ¶

1. Command name & plain English meaning

Shows **forward path hops** via TTL expiry—UDP/ICMP depending on implementation.

2. Most frequent production flags

- `-n` numeric - `-T` TCP SYN (if supported) - `-I` ICMP echo

3. Concrete command examples

```
tracert -n 203.0.113.10
tracert -T -p 443 api.bank.com
```

4. Interpreting the output (outage triage)

Stars `* * *` mid-path may be **ICMP rate-limit**, not outage. Last hop missing → target firewall.

5. Real-world production war story

Latency jump at hop 4 consistently—provider **hot potato** change; opened ticket with tracert proof.

Command reference: `tcptraceroute` ¶

1. Command name & plain English meaning

Traces path using **TCP packets**—useful when ICMP blocked but **443/tcp** allowed.

2. Most frequent production flags

- `-p PORT` — destination port - `-n` numeric

3. Concrete command examples

```
sudo tcptraceroute -n -p 443 api.bank.com
```

4. Interpreting the output (outage triage)

Path follows **stateful firewalls** that permit established flows. Stalls after SYN without SYN-ACK → ACL drop.

5. Real-world production war story

Partner “network open” claim disproved: `tcptraceroute -p 443` stopped at their edge; ICMP traceroute looked fine.

Command reference: `mtr -rwzbc 100` ¶

1. Command name & plain English meaning

Combines ping + traceroute with continuous statistics—best for intermittent loss/latency.

2. Most frequent production flags

- `-r` report mode - `-w` wide - `-z` show AS - `-b` show both host/IP - `-c 100` — 100 probes

3. Concrete command examples

```
mtr -rwzbc 100 api.bank.com
mtr -n -T -P 443 api.bank.com
```

4. Interpreting the output (outage triage)

Watch **Loss%** per hop (beware **ICMP bias**). **Worst** and **Last** RTT columns for drift.

5. Real-world production war story

Intermittent 2% payment failure: `mtr` showed 15% loss only on hop 6—provider fixed congested peering.

Command reference: `tcpdump` ¶

1. Command name & plain English meaning

Packet capture at the wire—ultimate ground truth for RST, retrans, TLS failures.

2. Most frequent production flags

- `-i eth0` — interface - `-nn` — no DNS/port names - `-vvv` — verbose - `-w file.pcap` — save - `-c N` — stop count - `-s0` — full snaplen

3. Concrete command examples

```
sudo tcpdump -i any -nn -vvv host 10.0.5.20 and port 443 -w /tmp/capture.pcap
sudo tcpdump -nn 'tcp[tcpflags] & tcp-syn != 0 and tcp[tcpflags] & tcp-ack == 0'
```

4. Interpreting the output (outage triage)

SYN retrans → no SYN-ACK. **RST** after SYN → ACL/port closed. **Zero-window** → receiver backlog. **TLS ClientHello without ServerHello** → middlebox or cipher mismatch.

5. Real-world production war story

Vendor denied sending RST: `tcpdump` showed `[R.]` from their IP within 2ms of SYN—ticket closed in our favor; firewall rule fixed.

Command reference: `ip addr` ¶

1. Command name & plain English meaning

Shows **IP addresses** per interface—from `iproute2` suite (replaces `ifconfig`).

2. Most frequent production flags

`addr show` — all `addr show dev eth0` — one NIC

3. Concrete command examples

```
ip -br addr
ip addr show dev eth0
```

4. Interpreting the output (outage triage)

Missing **UP** flag → link down. **Secondary IPs** for VIP/keepalived. **scope link** vs **global**.

5. Real-world production war story

Keepalived failover left `eth0:1` stale secondary—applications bound to wrong source IP; `ip addr` caught ghost VIP.

Command reference: `ip link` ¶

1. Command name & plain English meaning

Layer-2 interface state—up/down, MTU, promiscuous mode.

2. Most frequent production flags

`link show` `link set eth0 mtu 9000`

3. Concrete command examples

```
ip -br link
ip link show eth0
```

4. Interpreting the output (outage triage)

MTU mismatch causes black-hole TCP (small ping works, large transfers hang). `state DOWN` → cable/driver.

5. Real-world production war story

Jumbo frames enabled on host but not switch—large S3 uploads hung; `ip link` MTU 9000 vs path 1500; PMTUD fix.

Command reference: `ip route` ¶

1. Command name & plain English meaning

Routing table—where packets go next.

2. Most frequent production flags

`route show` `route get DST` — resolved path for one destination

3. Concrete command examples

```
ip route show table all
ip route get 203.0.113.55 from 10.0.1.5
```

4. Interpreting the output (outage triage)

Wrong **default via** → asymmetric routing. **Policy routing** tables (`ip rule`) common in multi-NIC servers.

5. Real-world production war story

Pod egress SNAT issue: `ip route get` from pod IP showed table **100** rule sending traffic to wrong gateway.

Command reference: `ip neigh` ¶

1. Command name & plain English meaning

ARP/NDP neighbor cache—L2 address for L3 next hop.

2. Most frequent production flags

`neigh show` — cache `neigh flush` — careful on prod

3. Concrete command examples

```
ip neigh show
ip -s neigh show dev eth0
```

4. Interpreting the output (outage triage)

STALE / REACHABLE normal. **FAILED** → ARP miss (wrong VLAN, host down). **Flapping REACHABLE** → loop/mac move.

5. Real-world production war story

Mystery partial connectivity: `ip neigh` showed **INCOMPLETE** for default gateway—bad switch port security.

Command reference: `ethtool -S eth0` ¶

1. Command name & plain English meaning

NIC hardware statistics—drops, errors, ring overruns—not visible in `ifconfig` alone.

2. Most frequent production flags

- `-S` — stats - `-g` — ring parameters - `-k` — offload settings

3. Concrete command examples

```
ethtool -S eth0 | egrep 'drop|err|miss'
ethtool -g eth0
```

4. Interpreting the output (outage triage)

Rising `rx_missed_errors` or `rx_fifo_errors` → buffer overrun at line rate. `tx_dropped` → qdisc/backpressure.

5. Real-world production war story

Payment spike: `rx_missed_errors` climbed with UDP syslog storm—moved logging off data NIC.

Command reference: `iptables -L -n -v` ¶

1. Command name & plain English meaning

Netfilter rules listing with counters—debug DNAT/SNAT/filter drops.

2. Most frequent production flags

- `-L` list - `-n` numeric - `-v` verbose counters - `-t nat` NAT table

3. Concrete command examples

```
sudo iptables -L -n -v
sudo iptables -t nat -L -n -v
```

4. Interpreting the output (outage triage)

Non-zero **DROP/REJECT counters** climbing during incident → rule hit. Compare `raw / mangle / filter / nat` tables.

5. Real-world production war story

“Random” API failures: `iptables -L -n -v` showed REJECT on NEW from pod CIDR after automation pushed wrong rule.

Command reference: `conntrack -L` ¶

1. Command name & plain English meaning

Inspect **connection tracking table**—critical when `nf_conntrack: table full` drops packets.

2. Most frequent production flags

- `-L` — list (heavy) - `-S` — stats - `-C` — count

3. Concrete command examples

```
sudo conntrack -S
grep conntrack /proc/sys/net/netfilter/nf_conntrack_count
```

4. Interpreting the output (outage triage)

Compare **count vs max**. High **TCP half-open** → SYN flood or scan. **Timeouts** too long → table churn slow.

5. Real-world production war story

Peak UPI traffic: kernel dropped packets; `conntrack -S` at 99% of `nf_conntrack_max`. Raised max and reduced `nf_conntrack_tcp_timeout_time_wait` after modeling.

Command reference: `bpftrace / BCC tcpretrans` ¶

1. Command name & plain English meaning

eBPF programs attach kernel tracepoints/kprobes for low-overhead production signals—retransmits, syscall latency, run queue delay.

2. Most frequent production flags

bpfftrace: `-e 'PROGRAM' inline script; count() , hist()` aggregations **BCC:** `/usr/share/bcc/tools/tcpspretrans , biolateness , runqlat`

3. Concrete command examples

```
sudo bpfftrace -e 'tracepoint:tcp:tcp_retransmit { @[comm] = count(); }'
sudo /usr/share/bcc/tools/tcpspretrans 5
sudo /usr/share/bcc/tools/runqlat 1 10
```

4. Interpreting the output (outage triage)

Spikes in `tcp_retransmit` by `comm` pinpoint blame without packet capture. `runqlat` ms buckets explain tail latency when CPU looks idle (runnable queue).

5. Real-world production war story

CRED-style p99 regression: `tcpspretrans` implicated one downstream dependency IP; `tcpdump` confirmed vendor-side loss—avoided week-long app profiling rabbit hole.

Advanced: eBPF for SRE (brief)

Tool	Use
<code>bpfftrace</code>	One-liners on tracepoints (<code>tcp_retransmit</code> , <code>runqlat</code>)
<code>bcc / libbpf tools</code>	<code>tcpspretrans</code> , <code>execsnoop</code> , <code>biolateness</code>
<code>perf</code>	CPU flamegraphs + off-CPU stacks

Example:

```
sudo bpfftrace -e 'tracepoint:tcp:tcp_retransmit { @[comm] = count(); }'
sudo /usr/share/bcc/tools/tcpspretrans 5
```

During outages, eBPF proves **where** time is spent without restarting apps with `strace` overhead.

Module 5: Top 30 SRE Linux & Networking Interview Questions

1. Step-by-step: What happens when you type `curl https://api.bank.com/v1/pay ?`

Answer (senior bar): 1. **Shell** parses command; **curl** loads CA bundle, resolves `api.bank.com` via glibc NSS → `/etc/resolv.conf` / stub resolver → recursive resolver → authoritative A/AAAA (possibly CNAME chain). 2. **Routing:** Kernel **FIB** picks egress iface/gateway; **ARP/ ip neigh** resolves next-hop MAC. 3. **Socket:** `socket()` → `connect()` triggers **TCP SYN** to :443 (or Happy Eyeballs if IPv6/IPv4). 4. **Handshake:** SYN → SYN-ACK → ACK; **congestion window** starts (slow start). 5. **TLS 1.3:** ClientHello → ServerHello → encrypted cert verify → Finished; **SNI** selects cert; **ALPN** negotiates HTTP/1.1 or h2. 6. **HTTP:** GET/POST over TLS record layer; server **accept queue** already consumed at listen socket during handshake burst. 7. **Response path:** NIC → TCP reassembly → TLS decrypt → curl `write()` to stdout; metrics show latency as DNS + TCP + TLS +

TTFB + download. 8. **Failure hooks:** DNS TTL staleness, SYN drop (LB), `CLOSE_WAIT` leak, cert expiry, MTU black hole, **HTTP 502** from upstream not TCP failure.

2. Load average vs CPU utilization—load 10 on 4 cores with 10% CPU?¶

Yes, possible. Load average includes **runnable + uninterruptible (D)** threads. Example: 40 threads stuck in `D` on NFS + 0.4 runnable cores $\approx$ load ~ 10 with **$\sim 10\%$ CPU busy**. Always triage with `top / ps`, `STAT`, `iostat`, `mount` stats—not CPU alone.

3. No space left on device but df -h shows 45% used?¶

Causes: (1) **inode exhaustion** (`df -i 100%`), (2) **quota**, (3) **unlinked open files** (`lsof +L1`), (4) **wrong filesystem** (full `/var` while checking `/`). **Fix:** delete files / rotate logs / restart holder / expand inodes (recreate fs) / fix quota.

4. TIME_WAIT VS CLOSE_WAIT ; 10,000 CLOSE_WAIT ?¶

TIME_WAIT: Local side closed first; waits **$2 \times \text{MSL}$** for stray segments—normal on high-churn clients. **CLOSE_WAIT:** Remote sent **FIN**; **application never `close()`** —**app bug/leak**, not kernel/network. **10k CLOSE_WAIT** → restart mitigates; fix code + FD limits + connection pool lifecycle.

5. ping vs traceroute ; ping OK but :443 hangs?¶

Ping tests **ICMP** reachability. **Traceroute** maps **path** (TTL). **Port 443** needs **TCP connect** + TLS + app. ICMP allowed + **TCP drop/filter** → ping success, curl hang. Use `tcptraceroute -p 443`, `nc -vz`, `curl -v`, `tcpdump`.

6. Intermittent 5s connection timeouts at peak—isolate DNS vs SYN vs conntrack vs thread pool?¶

Method: 1. Timestamp correlate **metrics** (p99 connect latency). 2. `dig +stats` latency vs `/etc/resolv.conf`. 3. `ss -s`, `tcpdump` SYN retrans count, `conntrack -S` fullness. 4. `strace -e trace=network -T` on app—where 5s spent? 5. **Thread pool** → app logs + `jstack`/profiler if connect is fast but request slow. 6. **Firewall** → `iptables -L -n -v`, `nf_conntrack` drops in `journalctl -k`.

7. User vs kernel space; duplicate process vs new program?¶

User space (Ring 3) apps; **kernel space** (Ring 0) privileged. **Syscalls** bridge them. `fork()` duplicates process; `execve()` loads new program image. `clone()` fine-grained (threads).

8. OOM Killer selection and protecting kubelet/sshd?¶

OOM uses `oom_score` heuristics (memory usage, runtime, root, child processes). Tune `/proc/PID/oom_score_adj` (`-1000` prevents kill for critical daemons on some policies). **Kubernetes:** QoS **Guaranteed**, `memory limits`, `system-reserved`, `kube-reserved`, `oom_score_adj` in kubelet config. **sshd:** `OOMScoreAdjust=-1000` in systemd unit.

9. When `ss` over `netstat` ; inspect listen backlog?

`ss` reads stats from `/proc/net/*` efficiently. `ss -ltn` shows **Recv-Q/Send-Q**; for listeners **Recv-Q** = completed connections waiting for `accept()` . Compare to `somaxconn` and app backlog.

10. tcpdump proving vendor sends TCP RST?

Capture on client/host: `tcpdump -i any -nn host VENDOR_IP and port 443 -w proof.pcap` . Show `[S]` then `[R.]` from **vendor IP** with no `[S.]` completion—or **RST after data**. Share **pcap + frame numbers**; correlate with **UTC NTP-synced** time.

11. Explain `SYN backlog` VS `accept queue` .

SYN backlog (`tcp_max_syn_backlog`) holds half-open connections during handshake. **Accept queue** (`min(backlog, somaxconn)`) holds fully established sockets until `accept()` . Drops manifest as **client SYN retrans** while CPU is idle.

12. What is head-of-line blocking in HTTP/1.1 vs HTTP/2?

HTTP/1.1: one slow response blocks connection reuse. HTTP/2 multiplexes streams on one TCP connection—still subject to **TCP HOL** on loss. HTTP/3 over QUIC solves transport HOL for web workloads.

13. How does PMTUD black hole happen?

ICMP **Fragmentation Needed** blocked while DF set → TCP sends full MSS forever with no progress. Fix: lower **MSS clamp** (`iptables -t mangle -TCPMSS`), enable ICMP, or reduce interface MTU consistently.

14. Difference between `softirq` and process CPU?

Network RX can burn **softirq** on ksoftirqd CPUs showing high `%hi` in `mpstat` while app CPU looks fine—check `/proc/softnet_stat` , RPS/RFS, NIC offload.

15. When is `tcp_tw_reuse` safe?

Modern kernels changed semantics; **do not tune by blog posts**. Measure `TIME_WAIT` counts, connection churn, and ephemeral port usage first. Prefer **more local ports, connection pooling, proper LB** over aggressive reuse.

16. Explain `SO_REUSEPORT` .

Multiple binders share load at **hash of 4-tuple**—reduces accept skew on multi-core. Misconfig can cause uneven workers (`htop` skew).

17. How do cgroups v2 limit memory and affect OOM?

memory.max triggers cgroup OOM killer before host OOM; pods die independently. SRE must set **requests/limits** and watch **memory.high** throttling.

18. Debug `Certificate verify failed` in production safely.¶

Compare **SNI**, **intermediate chain**, **clock skew**, **corporate MITM proxy**. Use `openssl s_client -connect host:443 -servername host -showcerts` from **same network namespace** as app.

19. What does `nf_tables` vs `iptables` mean for ops?¶

`nftables` backend on modern distros; `iptables-nft` translates rules. Counters and rule paths differ—validate automation targets correct backend.

20. How to prove asymmetric routing?¶

`tcpdump` on both directions, `ip route get`, `conntrack` entries. SYN arrives iface A, return via B → stateful firewall may drop.

21. Role of `systemd` socket activation?¶

Service starts on first connect—impacts **listen backlog** timing and FD inheritance; know `Accept=yes` units for superservers.

22. Explain `epoll` vs `select` for SRE.¶

High-connection APIs use **epoll** edge/level triggered. **Thousands of idle connections** → watch **FD ulimit** and `epoll_wait` latency under load.

23. What is TCP keepalive vs HTTP keep-alive?¶

TCP keepalive detects dead peers at L4; HTTP keep-alive reuses TCP for multiple requests—different timeouts; misaligned idle timeouts cause **RST surprises**.

24. How do you read `ss -i` TCP info?¶

Fields like **rtt**, **cwnd**, **retrans**—rising **retrans** with stable RTT suggests loss; RTT spike without retrans suggests bufferbloat/path change.

25. Diagnose DNS latency vs application latency.¶

`dig +stats` for Query time; app metrics with `trace DNS` span; if DNS 2ms but p99 500ms, look TCP/TLS/backend.

26. Safe production use of `strace`.¶

Attach briefly; `-c` summary mode; filter `-e trace=`; detach—heavy `strace` slows apps; prefer **eBPF** for long captures.

27. What triggers zombie processes?¶

Child exited, parent didn't `wait()`. Fix parent bug; zombies don't use memory but consume PIDs if unbounded.

28. Explain VFS and page cache for log-heavy apps.¶

Writes go to **page cache** marked dirty; `pdflush` flushes. Burst logging → **high dirty ratio**, `iowait`, latency— tune `/proc/sys/vm/dirty_*` or reduce sync logging.

29. Kubernetes networking: where do drops happen?¶

CNI overlay, **iptables/IPVS**, **kube-proxy**, **NetworkPolicy**, node **conntrack**, **CoreDNS**—isolate with `tcpdump -i any` in pod netns and `ss` on node.

30. Design an on-call runbook for total payment API outage (first 15 minutes).¶

1) Confirm blast radius & SLO burn 2) Check recent deploys/feature flags 3) `ss -s / conntrack` 4) **DNS/TLS/LB** health 5) **DB/Redis** dependencies 6) `tcpdump /vendor status` 7) comms + rollback decision 8) post-incident **blameless** timeline with evidence artifacts.

Module 6: Step-by-Step 14-Day Hands-On Study & Terminal Practice Plan¶

Day	Theory (read)	Terminal drills (do)	Success criteria
1	Module 1: rings, syscalls, /proc	<code>nproc</code> , <code>read /proc/cpuinfo</code> , <code>/proc/meminfo</code> , <code>/proc/sys/kernel/pid_max</code>	Explain Ring 0 vs 3 to a peer in 2 minutes
2	Process states, fork/exec	Spawn <code>sleep 300 &</code> , <code>ps</code> , <code>kill -STOP</code> , observe <code>T</code> state	Identify <code>D</code> vs <code>S</code> in <code>ps</code> man page
3	FDs, ulimits	<code>ulimit -n</code> , open many files script until error, fix with <code>LimitNOFILE</code>	Reproduce & fix EMFILE locally
4	Module 2 CPU	<code>mpstat -P ALL 1</code> , <code>pidstat -u 1</code> , stressing CPU	Correlate load vs CPU on stressed host
5	Memory & OOM	<code>free -w</code> , <code>vmstat 1</code> , tune <code>oom_score_adj</code> on test process	Predict OOM victim with <code>/proc/PID/oom_score</code>
6	Disk & inodes	Create many small files to exhaust inodes; fix with <code>df -i</code>	Recover without reboot
7	<code>strace</code> / <code>lsof</code>	Trace <code>curl</code> connect; find <code>+L1</code> deleted log scenario	Capture 5s connect delay in strace log
8	Module 3 TCP/DNS	Draw handshake; <code>dig +trace</code> on real domain	List socket states from <code>/proc/net/tcp</code>
9	<code>ss</code> / <code>tcpdump</code> localhost	Run <code>nc -l 9999</code> , connect, capture SYN/FIN	Explain Recv-Q on listener
10	Path MTU	<code>ping -s 1472 -M do</code> to external host	Detect PMTU issue or confirm end-to-end 1500
11	<code>mtr</code> / <code>tcptraceroute</code>	Compare ICMP vs TCP path to :443	Document hop with highest loss
12	Conntrack lab	Fill table in sandbox; observe kernel log message	Mitigate with <code>sysctl</code> + timeout tuning
13	systemd/journal	Break unit intentionally; restore with <code>journalctl -u</code>	Write 5-line postmortem from logs alone
14	Mock incident	Combine: DNS wrong + SYN drop script; triage in 30 min	Produce pcap + 1-page RCA

Sandbox setup (safe local practice)¶

```
# Network namespace lab (no root cloud needed on laptop)
sudo ip netns add sre-lab
sudo ip link add veth0 type veth peer name veth1
sudo ip link set veth1 netns sre-lab
# ... assign IPs, run tcpdump on veth0, curl from netns
```

Daily habit (30 min maintenance)¶

- One `tcpdump` exercise, one `ss -s` snapshot on a real server (read-only), one `dig +trace` for a dependency you own.

Appendix A: TCP state quick reference¶

Hex (/proc/net/tcp)	State
01	ESTABLISHED
02	SYN_SENT
03	SYN_RECV
04	FIN_WAIT1
05	FIN_WAIT2
06	TIME_WAIT
07	CLOSE
08	CLOSE_WAIT
0A	LISTEN

Appendix B: Recommended reading¶

- Site Reliability Engineering (Google) — Ch. 6 Monitoring, Ch. 28 Accelerating SREs
- Systems Performance (Gregg) — CPU, memory, network chapters
- TCP/IP Illustrated, Volume 1 (Stevens) — protocol ground truth
- Linux man pages: `socket(7)` , `tcp(7)` , `ip(7)` , ``systemd.system(5)`

Document version 1.0 — generated for production SRE interview and on-call preparation.